SoC Boot-to-Runtime Verification

RISCFORGE 1TOPS

System-Level Testbench | VERIF-SOCTB-v3.0

Aim

The aim is to show, in one continuous simulation, that the SoC can take a firmware image out of an external SPI flash chip and run it on the RV32IM core.

The image is transferred over the SPI bus into the vendor flash model and read back by the bootloader; no backdoor write is used on either SRAM. Because the program cannot reach its final state unless every link in that chain carried real data, a completed run establishes each of those links individually.

Note

This is a system integration test, not an ISA compliance test. It exercises each path between the core and the SoC once. Instruction coverage is deliberately small — see Section 8.

Setup

Tools

Item	Value
Simulator	Cadence Xcelium 22.09-s003
Top module	`soc_timer0_boot_run_tb`
Filelist	`rtl/soc_timer0_boot_run.f`
Command	`xrun -f rtl/soc_timer0_boot_run.f`
Working directory	`RTL_SOC/` — required, all image paths are relative to it
Clock	100 MHz, 10 ns period
Access mode	`-access +rwc`, needed for the testbench probes
Waveforms	Off by default; add `+WAVES` to enable

Important

Waveforms are off by default on purpose. Cold boot alone takes about 122,000 cycles, so dumping the whole hierarchy every run produces an enormous database. Use `+WAVES` only when debugging.

IPs and Modules Used

The testbench elaborates 1197 module instances across 91 unique modules. The ones that matter for this test are:

Module	Origin	Role Here
`s25fs512s`	Infineon vendor model	Behavioural SPI NOR flash holding the firmware image. Driven over a real SPI bus.
`bootloader`	In-house	Boot FSM and reset controller. Talks to the SPI IP over APB and to the SRAMs over AXI.
`apb_spi_master`	PULP-derived	Moves the image bits during boot.
`axi_lite_xbar`	PULP	2-to-3 AXI-Lite crossbar.
`axi_lite_to_apb`	PULP	AXI-Lite to 6-slave APB bridge.
`apb_timer`	PULP-derived	The MMIO target. Timer 0's register file is what the program reads and writes.
`top` (RV32IM)	In-house	The processor under test.
`hybrid_mem_interface`	In-house	Sends core data accesses to DSRAM or to MMIO.
`instr_fetch_demux`	In-house	Sends each fetch to the BootROM or to ISRAM.
`boot_rom`	In-house	16-word reset-vector ROM.
`imem_sram_wrapper`	In-house	32 KB instruction SRAM.
`sram_sp`	In-house	32 KB data SRAM.
`sram_mux`, `axi_lite_sram_if`	In-house	SRAM arbitration and AXI bridging.

I2C, UART, GPIO and Timer 1 are also elaborated but never touched. They are present because the testbench builds the complete subsystem rather than a cut-down version.

Building the Flash Image

The image is built offline by `ips/mem_files/gen_flash_image.py`, which reuses the vendor's own converter `spi_unified_mem_instr_data.py`. Two word-oriented hex files are merged into one little-endian byte stream starting at flash address 0.

File	Role	Contents
`riscforge_timer0_test.hex`	Input	11 instruction words
`riscforge_timer0_dmem.hex`	Input	4 data words
`riscforge_timer0_flash.mem`	Output	60 bytes, little-endian, from flash address 0

Listing 1: Regenerating the image

```
cd ips/mem_files && python gen_flash_image.py
```

The flash model is loaded through its `mem_file_name` parameter, not through a backdoor task, so the simulated chip holds exactly the bytes the converter produced.

The byte order survives the whole trip. Word 0 is `0x400010B7`. It becomes the flash bytes `B7 10 00 40`. SPI sends `B7` first, which lands in `prdata[31:24]`. The bootloader's RX-FIFO swap puts it back together as `0x400010B7`.

Data Path

Boot — Image Into Memory

The core is held in reset for all of this.

Step	Stage	What Happens
1	`s25fs512s` flash model	Holds the 60-byte image. Read with opcode `0x03`, standard SPI mode, `CLKDIV = 4`
2	`apb_spi_master`	Shifts the bits in; the bootloader FSM configures and triggers it over APB
3	Bootloader RX FIFO read	One 32-bit word read back over APB, bytes swapped into program order
4	Bootloader AXI-Lite master	Issues the SRAM write with `aw.prot = 3'b001` (privileged)
5	`axi_lite_xbar` SLV[1]	Decodes the destination address
6	MST[0] → `axi_lite_sram_if` (`PRIV_WRITE_ONLY`) → `sram_mux`	ISRAM, words 0–10
7	MST[1] → `axi_lite_sram_if` → `sram_mux`	DSRAM, words 11–14

The split point is `NUM_INSTR_WORDS`. The bootloader writes that many words starting at the ISRAM base, then switches its destination pointer to the DSRAM base for the rest.

Runtime — Three Separate Paths

Once the core is released, three different paths get used. The test is built so each one has its own visible consequence, so a failure tells you which path broke.

Path	Core Port	Route	Endpoint
Instruction fetch	`imem_sram_*`	`instr_fetch_demux`	BootROM `0x0000_0000` or ISRAM `0x0001_0000`
Data, direct	`lsu_mem_*`	`hybrid_mem_interface` → `sram_mux` port 0	DSRAM `0x0002_0000`
Data, MMIO	`lsu_mem_*`	`hybrid_mem_interface` → AXI-Lite → crossbar SLV[0] → MST[2] → `axi_lite_to_apb`	Timer 0, APB[1]

The direct data path is combinational into the SRAM and costs one stall cycle on reads. The MMIO path costs roughly three to four cycles each way.

The Test Program

BootROM — Two Instructions

These live in `boot_rom.sv`, loaded from `riscforge_bootrom.hex`. The core starts here.

Address	Encoding	Instruction	What It Does
0x0000_0000	0x00028137	lui x2, 0x28	Puts 0x0002_8000 into x2, setting the stack pointer to the top of DSRAM
0x0000_0004	0x7FD0F06F	jal x0, +0xFFFC	Jumps to 0x0001_0000, the start of ISRAM, where the loaded program lives

Main Program — Eleven Instructions

Loaded into ISRAM by the bootloader.

Address	Encoding	Instruction	What It Does
0x10000	0x400010B7	lui x1, 0x40001	Loads the Timer 0 base address 0x4000_1000 into x1
0x10004	0x00900113	addi x2, x0, 9	Puts the value 9 into x2. This is the timer control word: bit 0 enables the timer, bits 5:3 set the prescaler to 1
0x10008	0x0020A223	sw x2, 4(x1)	**MMIO write.** Stores 9 into TIMER_CTRL at 0x4000_1004, which should turn the timer on
0x1000C	0x0040A183	lw x3, 4(x1)	**MMIO read.** Reads TIMER_CTRL back into x3. Should return 9
0x10010	0x00020237	lui x4, 0x20	Loads the DSRAM base address 0x0002_0000 into x4
0x10014	0x00310663	beq x2, x3, +12	Compares what was written with what was read. If they match, jumps forward to the pass path
0x10018	0x00000293	addi x5, x0, 0	Fail path. Puts 0 in x5
0x1001C	0x0080006F	jal x0, +8	Skips over the pass path
0x10020	0x00100293	addi x5, x0, 1	Pass path. Puts 1 in x5
0x10024	0x00522023	sw x5, 0(x4)	**Direct SRAM write.** Stores the result into DSRAM word 0
0x10028	0x00000063	beq x0, x0, 0	Branches to itself. The program parks here forever

Expected Result

If everything works, the program takes the pass path. The end state should be:

Location	Expected Value	Meaning
x1	0x40001000	Timer base loaded
x2	0x00000009	Control word built
x3	0x00000009	**Read back the same value that was written**
x4	0x00020000	DSRAM base loaded
x5	0x00000001	Pass path taken
Timer 0 TIMER_CTRL	0x00000009	The write really reached the peripheral
Timer 0 TIMER_VAL	non-zero	The timer is actually running
DSRAM[0]	0x00000001	Result stored over the direct path
Final PC	0x00010028	Parked at the self-branch

Why This Program

Every instruction is there for a reason.

- **The `sw` then `lw` pair is a round trip.** A write that quietly fails looks the same as a read that quietly fails. But if you write 9 and read back 9, both directions of the whole AXI and APB fabric must have worked.
- **The `beq` turns the bus result into control flow.** If the fabric is broken, the program takes a different branch instead of just having one wrong register. That is much easier to spot.
- **The final `sw` uses the other data path.** The pass/fail answer travels over the direct SRAM link, which is independent of the MMIO link being tested. So the verdict cannot be corrupted by the thing it is judging.
- **The parking branch gives a stable end state** that the testbench can check.

Data Image

Four words are placed in DSRAM by the bootloader. Word 0 is the result slot. Words 1 to 3 are never touched by the program, so they show the data half of the boot load survived into runtime.

DSRAM Word	Value	Purpose
0	`0xDEADBEEF`	Result slot. The program overwrites it. The testbench watches for this change to know the program has finished
1	`0xCAFEBABE`	Untouched marker
2	`0x12345678`	Untouched marker
3	`0xA5A5A5A5`	Untouched marker

Test Cases

The testbench runs in five phases. Later phases depend on earlier ones, so a boot timeout stops the run instead of producing a pile of meaningless failures.

Phase 0 — Is the Image Really in the Flash?

Reads the image back out of the flash model before boot starts. This checks that the file was found, that the load parameter took effect, and that the byte order the converter produced reassembles into the right words.

Without this, a failed load shows up much later as an ISRAM full of `0xFFFFFFFF`, which is far harder to trace back.

Phase 1 — Does Cold Boot Finish?

Releases `hard_rst_ni` and waits for `boot_done_o`. Checks that all resets were low beforehand, that the SPI chip select actually toggled (so real SPI traffic happened), and that the bootloader ended up in its done state.

Phase 2 — Are the Resets Sequenced Correctly?

Checks that all three internal reset domains were released, that the SPI reset came out before the core reset, and that the core and peripherals came out together.

Phase 3 — Did the Image Land in the Right Places?

Checks that the 11 instruction words are in ISRAM, the 4 data words are in DSRAM, and ISRAM word 11 is still zero. That last one matters: it proves the clear pass ran and the image did not overrun its region.

This phase also prints write-path counters, so an ISRAM problem can be located without opening a waveform:

Listing 2: Write-path counters, healthy run

```
[TB] -- Boot write-path counters --
[TB]   ISRAM writes reaching the macro : 8203  (expected 8203)
[TB]   DSRAM writes reaching the macro : 8196  (expected 8196)
[TB]   ISRAM AW handshakes at bridge   : 8203
[TB]   ISRAM error responses (SLVERR)  : 0
[TB]   last aw.prot seen by the gate   : 3'b001
[TB]   >> ISRAM write path OK: every write reached the macro, no errors.
```

The numbers are exact, so they check themselves: 8192 clear writes plus 11 image words is 8203 for ISRAM, and 8192 plus 4 is 8196 for DSRAM.

Phase 4 — Did the Program Run Correctly?

Waits for `DSRAM[0]` to change from its marker value, then checks the end state listed in Section 4.3: the BootROM's effect, the program's registers, the timer's own registers, the DSRAM result, the untouched markers, and the final PC.

Note

The `x2` check needs care. The BootROM sets `x2` to the stack pointer, but the program's second instruction reuses `x2` as a scratch register and overwrites it with 9. So reading `x2` at the end tells you nothing about whether the BootROM ran. The testbench instead watches the register file's write port and captures the *first* write to `x2`. That first write is the BootROM's; every later one belongs to the program.

What Was Proven

The run completed with every check satisfied. Taken together, these are the capabilities the result establishes:

Capability	Evidence
The SoC can boot from external flash.	Real SPI traffic moved 15 words out of the vendor flash model into on-chip memory, using the real bootloader and the real SPI IP.
The image is split correctly.	11 instruction words landed in ISRAM at `0x0001_0000`; 4 data words landed in DSRAM at `0x0002_0000`; nothing overran.
The reset controller sequences correctly.	SPI released early so the SPI IP was usable during boot; core and peripherals released together only after the image was in place.
The core starts at the right address.	The BootROM's `lui` retired with `0x0002_8000`, proving the reset vector, the ROM contents and the ROM's reset behaviour are all correct.
Instruction fetch works through the demux.	The core fetched from the BootROM, jumped, and continued fetching from ISRAM — so both fetch targets and the switch between them work.
The direct data path to DSRAM works.	The program's final store changed `DSRAM[0]` from its marker to the result value.

Capability	Evidence
The MMIO path works in both directions.	The program wrote 9 to Timer 0's control register and read 9 back. Timer 0's own register holds 9 and the timer counted up, confirming at the peripheral end that the write physically arrived. This exercises the core, the hybrid interface, AXI-Lite, the crossbar, the APB bridge and the timer.
The loaded data survives into runtime.	The three untouched markers still held their values after the program finished.
The program terminates cleanly.	The PC settled at `0x0001_0028`, the self-branch, and stayed there — not frozen mid-pipeline.

Note

The result readback is the strongest single piece of evidence. `x3` could only contain 9 if the store reached the timer register and the load brought its contents back. No partial failure produces that value.

Timing

Event	Cycle	Comment
`hard_rst_ni` released	9	
SRAM clear finished	81,930	16384 serialised AXI writes
Flash power-up finished	111,930	30,000-cycle wait
SPI image fetch finished	121,627	About 646 cycles per word
`boot_done_o` asserted	121,634	
BootROM `lui` retired	121,637	4 cycles after the core was released
Program result stored	121,658	24 cycles of program execution

Note

Cross-check on the 24-cycle runtime. The testbench reports that `x2` was written five times. The program writes it once and the BootROM writes it once, which is two. The three extra writes are the `addi x2` instruction sitting in the writeback register while the following `sw` waits on the MMIO bus. Three extra writes means three stall cycles on that store — so the bus wait is real and measurable, without a waveform.

Defects Found and Fixed

Bring-up turned up five defects. Every one compiled and elaborated cleanly, and four of the five let the system appear to boot successfully. They are recorded here because they are the kind of failure that static checking does not catch.

Defect 1 — ISRAM Writes Silently Rejected

What happened

The image never reached instruction memory. ISRAM read back as all NOPs, so the core came out of reset and executed thousands of NOPs instead of the program. DSRAM, meanwhile, was filled perfectly.

Why

ISRAM is protected. Its AXI bridge is instantiated with `PRIV_WRITE_ONLY = 1`, which means it only accepts writes marked as privileged. The AXI-Lite protection field `aw.prot` carries that marking: bit 0 set means privileged.

The bootloader was driving `aw.prot` to zero. So every write it sent to ISRAM was unprivileged, and the bridge rejected all of them — 8192 clear writes and 11 image words, 8203 in total. DSRAM has no such gate, which is why it worked and ISRAM did not. That asymmetry was the clue that eventually identified the cause.

Fix

Set `axi_req_o.aw.prot = axi_pkg::prot_t'(3'b001)` in the bootloader's AXI request assembly, so its SRAM writes are marked privileged and pass the gate.

Important

Three copies of `bootloader.sv` exist in the tree and the copy being compiled was not the copy being read, so repeated inspection of the wrong file kept indicating the code was correct. Duplicated RTL should be consolidated to a single source.

Defect 2 — Core Deadlocked on Its First MMIO Access

Symptom

The core froze permanently at `PC = 0x0001_0018` with `x3`, `x4` and `x5` never written. The store at `0x1_0008` was still sitting in the MEM stage.

What the states mean

The write FSM in `hybrid_mem_interface` has four states. Which channel each waiting state is responsible for is defined by the valid outputs it drives:

Listing 3: Valid outputs define the meaning of each state

```
m_axi_awvalid = (axi_write_state == AXI_WRITE_IDLE && axi_req_active && mem_we)
              || (axi_write_state == AXI_WAIT_ADDR);
m_axi_wvalid  = (axi_write_state == AXI_WRITE_IDLE && axi_req_active && mem_we)
              || (axi_write_state == AXI_WAIT_DATA);
m_axi_bready  = (axi_write_state == AXI_WAIT_RESP);
```

`AXI_WAIT_ADDR` holds `awvalid` high, so it means *the address has not been accepted yet*. `AXI_WAIT_DATA` holds `wvalid` high, so it means *the data has not been accepted yet*. Each must therefore wait on its own channel's ready.

What was wired

The transition out of `AXI_WRITE_IDLE` was correct. The two waiting states were not — each polled the opposite channel:

Listing 4: Before (wrong) and after (correct)

```
// BEFORE
AXI_WAIT_ADDR: if (m_axi_resp_i.w_ready)  next_axi_write_state = AXI_WAIT_DATA;
AXI_WAIT_DATA: if (m_axi_resp_i.aw_ready) next_axi_write_state = AXI_WAIT_RESP;

// AFTER
AXI_WAIT_ADDR: if (m_axi_resp_i.aw_ready) next_axi_write_state = AXI_WAIT_RESP;
AXI_WAIT_DATA: if (m_axi_resp_i.w_ready)  next_axi_write_state = AXI_WAIT_RESP;
```

Why it deadlocked

The crossbar is configured `CUT_ALL_AX`, which puts a spill register on AW but not on W. On the first cycle of the store that register is empty, so `aw_ready` is high, while the W path through `axi_lite_to_apb` is not yet ready, so `w_ready` is low. The FSM sampled `{aw_ready, w_ready} = 2'b10` and moved to `AXI_WAIT_DATA`.

In `AXI_WAIT_DATA` the FSM kept `wvalid` high, which was correct, but waited on `aw_ready`. That signal was now low, because the AW spill register held the accepted address and would not drain until the write transaction completed at the APB slave — which required the W beat that the FSM was refusing to advance. Each side waited on the other:

Side	What It Is Waiting For
Write FSM	Holds `wvalid` high and waits for `aw_ready` to return high
Crossbar	`aw_ready` stays low because the AW spill register is full, and it cannot drain until the write completes at the APB slave
Write completion	Requires the W beat, which the FSM never sends because it never leaves `AXI_WAIT_DATA`

`axi_wr_busy` therefore stayed asserted forever, holding `mem_stall` high and freezing the pipeline.

Defect 3 — Transactions Issued to Address Zero

What was wired

Four registers carried the AXI payload:

Listing 5: The payload registers, declared at hybrid$_{m}em_{i}nterface.sv : 83 - 90$

```
logic [31:0] m_axi_awaddr;
logic [31:0] m_axi_wdata;
logic [3:0]  m_axi_wstrb;
logic [31:0] m_axi_araddr;
```

They were loaded inside the `always_ff`, in the `AXI_WRITE_IDLE` arm:

```
AXI_WRITE_IDLE: begin
  if (axi_req_active && mem_we) begin
    m_axi_wstrb  <= mem_be;      // non-blocking: visible at T+1
    m_axi_awaddr <= mem_addr;
    m_axi_wdata  <= mem_wdata;
  end
end
```

and the request struct was driven from those registers:

```
m_axi_req_o.aw.addr = m_axi_awaddr;   // registered
m_axi_req_o.w.data  = m_axi_wdata;    // registered
m_axi_req_o.ar.addr = m_axi_araddr;   // registered
```

The mismatch

The valid signals were combinational — `m_axi_awvalid` asserts in the *same* cycle the request arrives. The payload registers, being non-blocking assignments, only take their new value at the *end* of that cycle.

So during the accepting cycle the AW channel presented `awvalid = 1` alongside `m_axi_awaddr` still holding its previous contents — `32'h0000_0000` after reset. The crossbar's spill register accepts immediately when empty, which is exactly the case on the first access, so the transaction was latched carrying address zero.

Address zero matches no entry in `xbar_addr_map` (ISRAM starts at `0x0001_0000`), so the transaction was routed to `axi_err_slv` instead of the timer.

Fix

Drive the payload straight from the LSU bus and delete the register dependency:

```
m_axi_req_o.aw.addr = mem_addr;
m_axi_req_o.w.data  = mem_wdata;
m_axi_req_o.w.strb  = mem_be;
m_axi_req_o.ar.addr = mem_addr;
```

This is only legal once Defect 4 is also fixed: `mem_stall` must freeze the pipeline so that `mem_addr` and `mem_wdata` stay stable for the whole transaction.

Defect 4 — The Core Never Waited for the Bus

What `mem_stall` is for

The MEM stage assumes an access finishes in the cycle it is issued. Anything slower — a registered SRAM read, or an MMIO transaction crossing AXI and APB — must hold the pipeline until the data is genuinely available. `mem_stall` is that hold signal, and it is the only back-pressure the memory system has.

It travels five modules and ends at five separate consumers:

Step	Signal	Module and Effect
1	`mem_stall`	`hybrid_mem_interface` output
2	`lsu_mem_stall`	Through `top.v` into `memory_cycle`
3	`core_stall`	`ls_unit.v:50` — `core_stall = mem_req & mem_stall`
4	`core_stall_gated`	`memory_cycle.v:184` — gated so a misaligned access traps instead of stalling

From there it reaches five consumers, all of which must act in the same cycle:

Location	Effect
`memory_cycle.v:255`	Holds the MEM/WB pipeline register
`hazard_unit.v:170`	`StallF` — freezes the PC
`hazard_unit.v:171`	`StallD` — freezes IF/ID
`hazard_unit.v:173`	`StallE` — freezes ID/EX
`hazard_unit.v:175`	`StallM` — freezes EX/MEM

`mem_stall_out` carries it from `memory_cycle` to `hazard_unit` via `top.v:625`.

If `mem_stall` is not high in the cycle the request is issued, none of those five holds happen and the access retires immediately whether or not the bus has responded.

What was wired

`axi_w_stall` and `axi_r_stall` were declared as `logic` and assigned inside the `always_ff`:

Listing 6: hybrid$_{m}em_{i}nterface.sv : 234 - - - aflop, notawire$

```
AXI_WRITE_IDLE: begin
  if (axi_req_active && mem_we) begin
    ...
    axi_w_stall <= 1'b1;        // non-blocking: high only from T+1
  end
end
```

and the aggregation read them directly:

```
mem_stall = sram_stall_needed | axi_w_stall | axi_r_stall;
```

Consequence

At cycle T, when the LSU asserted `mem_req` with an MMIO address, both flops still held their old value, so `mem_stall` evaluated to 0. `ls_unit` computed `core_stall = mem_req & mem_stall = 0`, `core_stall_gated` was 0, and `memory_cycle` latched MEM/WB on that same edge. The store retired before its transaction had begun.

For the load the damage was visible in the result: MEM/WB captured `lsu_core_rdata`, which `ls_unit` had formatted from `mem_rdata`, which for an MMIO address is `axi_rdata_reg` — a register that had never been written. That is why `x3` read `0x0000_0000` instead of 9.

The whole eleven-instruction program finished in 24 cycles, which is arithmetically impossible for two AXI-to-APB round trips and was the clue that identified this defect.

Fix

Derive the stall combinationally from the FSM state, so it is correct in the cycle the request is made:

```
wire axi_wr_done = (axi_write_state == AXI_WAIT_RESP) && m_axi_resp_i.b_valid;
wire axi_rd_done = (axi_read_state  == AXI_WAIT_READ) && m_axi_resp_i.r_valid;

wire axi_wr_busy = axi_req_active && mem_we && !axi_wr_done;
wire axi_rd_busy = axi_req_active && mem_re && !axi_rd_done;

mem_stall = sram_stall_needed | axi_wr_busy | axi_rd_busy;
```

`mem_err` needed the same treatment. The registered `axi_w_err` and `axi_r_err` would arrive after the core had already retired the access, so a bus error was never seen by the trap logic.

`mem_rdata` additionally forwards `m_axi_resp_i.r.data` combinationally on the cycle `r_valid` arrives, because that is the same cycle the stall drops and MEM/WB latches the result.

Coverage

What this test does not cover: -

Area	Status
M-extension	Not exercised. `rv32m_mul.v` also has unresolved port-width mismatches reported at elaboration. This is the biggest gap.
Sub-word accesses	`LB`, `LH`, `LBU`, `LHU`, `SB`, `SH` untested. Only `LW` and `SW` are covered.
Traps and `MRET`	No exception is deliberately raised, so the trap controller is untested.
Interrupts	`mstatus.MIE` is never set, so no interrupt is ever taken.
CSR instructions	The `CSRRW`, `CSRRS` and `CSRRC` group is not executed.
Warm boot	Only cold boot is exercised; `soft_rst_ni` stays high throughout.
I2C, UART, GPIO, Timer 1	Elaborated but never accessed.